Project overview + 60-second answer

What to say first, and the six points to remember

THE WHOLE PROJECT IN ONE SENTENCE

I designed and tested an ESP32-based temperature-sensing prototype, analyzed real sensor data in Python, and converted the circuit into an editable KiCad schematic, routed PCB and 3D design.

SIX KEY POINTS

01 Purpose	Temperature sensing and validation using an ESP32 and NTC thermistor.
02 Circuit	9.82 kΩ divider resistor, 990 Ω ADC series resistor, 100 nF filter capacitor.
03 Firmware	GPIO34 ADC1; 16-reading average; temperature calculated from NTC voltage.
04 Python	Live serial dashboard, CSV recording, analysis and graphs.
05 Testing	641 stable samples; 0.0412°C sample SD; heating/cooling response demonstrated.
06 PCB	50 × 35 mm KiCad PCB with DRC/parity checks and 3D visualization.

MEMORIZE THIS - ABOUT 60 SECONDS

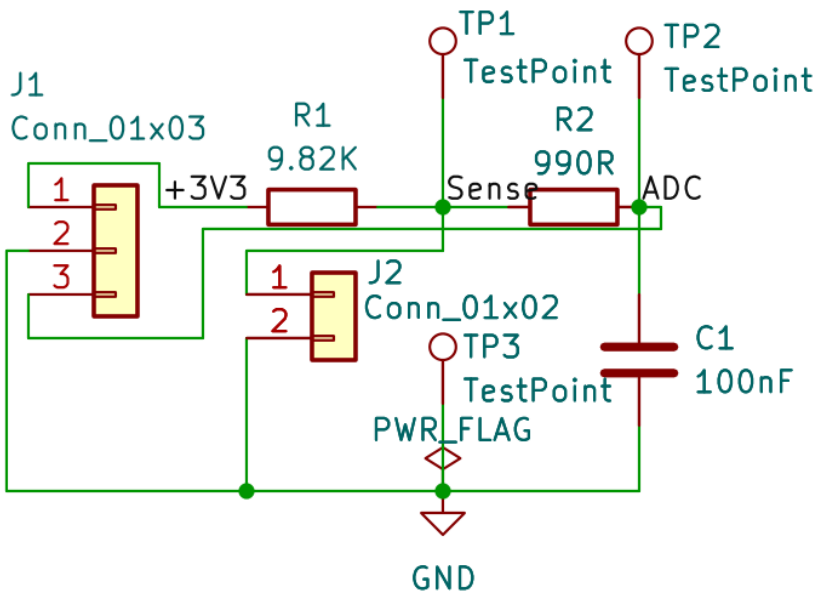
SAY IT NATURALLY, NOT WORD-FOR-WORD

“One of my engineering projects is called ThermoBench. It is an ESP32-based temperature-sensing and validation system. I built a breadboard circuit using resistors, a capacitor and an NTC thermistor. The thermistor changes resistance with temperature, which changes the voltage read by GPIO34. I configured firmware to collect voltage readings and estimate temperature, and used Python for live monitoring and CSV data analysis. I checked the circuit with a multimeter and recorded hardware tests. The steady-state test had a 0.041°C standard deviation, and I demonstrated warming and cooling response. After testing the prototype, I created the KiCad schematic, routed a custom PCB and generated 3D views. The PCB passed the configured design checks, but has not been manufactured yet. The project strengthened my skills in sensors, embedded code, troubleshooting and validation.”

How the circuit works

Show the real schematic, explain the function of each part

ACTUAL KICAD SCHEMATIC



COMPONENT TALKING POINTS

R1 | 9.82 kΩ	Upper half of the divider; pulls SENSE toward 3V3.
NTC | nominal 10 kΩ	External sensor at J2; resistance falls as temperature rises.
R2 | 990 Ω	Series resistor between SENSE and the ESP32 ADC node.
C1 | 100 nF	Filters some high-frequency electrical noise at the ADC input.
J1 and J2	J1 = ESP32 power / ground / GPIO34. J2 = thermistor connection.
TP1 / TP2 / TP3	Places to probe SENSE, ADC and GND using a multimeter.

WHAT TO SAY

“When the thermistor warms up, its resistance drops. In this divider arrangement the measured voltage drops too. The ESP32 reads the filtered ADC voltage, and the firmware calculates a temperature estimate. I used the test points to compare the hardware readings with a multimeter.”

Software and firmware

Explain the complete sensor-to-report workflow

SIGNAL AND DATA FLOW

NTC thermistor
Temperature changes R

Divider + RC filter
Converts R to voltage

ESP32 GPIO34
Samples and averages ADC

USB serial
Transmits one row / 200 ms

Python dashboard
Live chart and CSV recording

Analysis
Plots and repeatability checks

SOFTWARE - SIMPLE DESCRIPTIONS

Arduino IDE / C++	Configured ESP32 ADC acquisition and serial output.
Python	Displayed readings, saved CSV data and generated validation plots.
KiCad Schematic Editor	Drew and checked the circuit connectivity.
KiCad PCB Editor	Assigned footprints, routed copper, checked design rules.
KiCad 3D Viewer	Inspected the proposed assembled board visually.
Multimeter	Measured rail and test-point voltages to verify electrical behavior.

FIRMWARE BEHAVIOR

Every 200 ms: average 16 calibrated-millivolt API readings on GPIO34; convert voltage to thermistor resistance and model temperature; classify the reading; output ms,mv,temp_c,status over serial.

> **WHAT TO SAY**
>
> “I combined embedded acquisition with desktop analysis. The ESP32 sends a time-stamped voltage and temperature estimate, and Python records the raw results so I can graph them and check repeatability instead of relying on a single reading.”

Actual hardware validation

Real CSV captures, not simulated temperatures

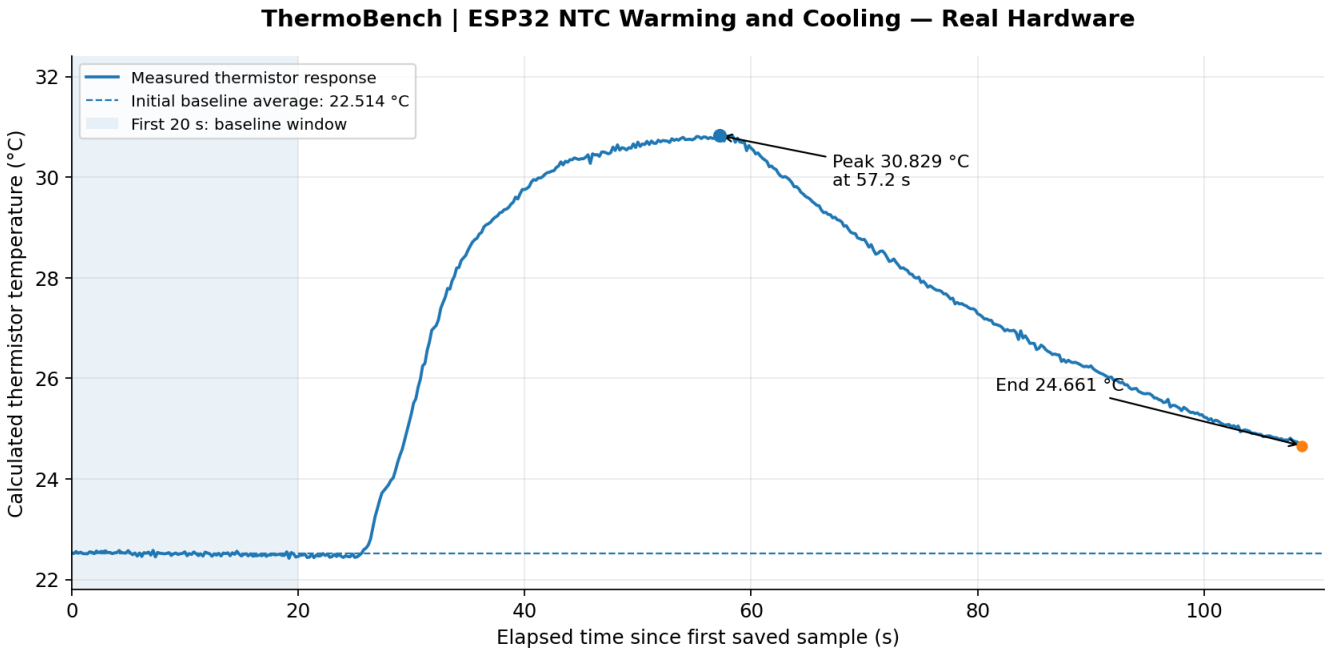
TWO HARDWARE TESTS TO DESCRIBE

641	0.0412°C	30.829°C
stable readings	temperature sample SD	warm-up peak

Stable run: 128.0 seconds at 5 Hz (200 ms interval). Mean model temperature 22.377°C. Sample standard deviation 0.0412°C versus a project-defined ≤0.15°C repeatability screen - pass.

Warming and cooling: 544 real samples over 108.6 seconds. Initial 20-second mean 22.513°C; peak 30.829°C at 57.2 seconds; final reading 24.661°C. Full return to baseline was not demonstrated.

ACTUAL MEASURED RESPONSE



544 HARDWARE readings | 5.00 Hz | No missing intervals or fault states | Temperature not reference-calibrated

WHAT TO SAY

“I first kept the sensor still and collected 641 readings to evaluate consistency. The sample standard deviation was 0.041°C, below my 0.15°C screening target. I then warmed the thermistor, watched the estimated temperature rise to about 30.8°C, and observed it fall as the sensor cooled. The data shows response and repeatability, but not calibrated absolute accuracy.”

Troubleshooting and limitations

What you investigated and what you would improve

A REAL ENGINEERING CHALLENGE - DMM VERSUS ADC

~1.58 V	~1.609 V	~29 mV
multimeter observation	ESP32 ADC observation	difference, approximate

In an earlier baseline check, the multimeter and ESP32 ADC did not exactly agree. The voltage offset is a useful troubleshooting story, but it does not establish a final calibrated ADC error because the measurement conditions and reference uncertainty need a controlled repeat.

> **INTERVIEW ANSWER - TROUBLESHOOTING**
>
> “I noticed the ESP32 voltage and my multimeter were not identical. I checked the supply and test points, and I used a fixed resistor to create a stable input. That helped separate circuit wiring and measurement consistency from ADC calibration. My next improvement would be a controlled comparison against a reference and a correction method if the error is repeatable.”

MY TROUBLESHOOTING PROCESS

1 Define the symptom	What is wrong? Voltage, output, noise or missing data?
2 Measure the actual circuit	Check 3V3, TP1, TP2 and ground with a multimeter.
3 Isolate one variable	Substitute a known fixed resistor; inspect GPIO34 and ADC settings.
4 Compare predicted and observed	Check divider calculations against DMM and serial outputs.
5 Record and retest	Save raw CSV, change one variable, repeat the measurement.

WHAT IS STILL UNVERIFIED

Do not claim: ±0.041°C accuracy, certified temperature calibration, hardware fault detection, or successful operation of a fabricated PCB. The archived firmware also may not match the exact binary used for the recorded runs.

Next improvements: compare temperature to a traceable reference, verify ADC calibration, check fault-state operation, measure real component sizes, fabricate the PCB, and repeat the hardware tests on the assembled PCB.

Interview questions and answers

Eight questions you can practice out loud

SHORT ANSWERS

Why use an ESP32?	It provides analog inputs, is easy to program, and streams readings to my computer.
Why use an NTC thermistor?	Its resistance changes with temperature, so it is useful for practicing analog sensing and calibration concepts.
Why average 16 samples?	Averaging can reduce some random ADC noise and produce more consistent readings.
What is GPIO34?	It is an ADC1 input on the ESP32 that I use to measure the analog voltage.
Why use KiCad?	To capture the schematic, assign physical footprints, route the PCB, run checks and inspect it in 3D.
What is DRC?	Design Rules Check - KiCad checks selected layout rules such as clearances and connectivity.
Did you build the PCB?	I built and tested the breadboard circuit. I designed and software-checked the PCB, but have not had it fabricated.
What would you improve?	I would calibrate the readings against a reference, test fault conditions, and physically build and verify the PCB.

WHAT THIS PROJECT SHOWS AN EMPLOYER

Hands-on electronics: breadboarding, resistor selection, sensor measurement and multimeter use.
Embedded systems: GPIO34, ADC acquisition, sampling, signal conversion and serial communications.
Test engineering: stable-condition experiments, repeatability, traceable CSV data and documented limitations.
Hardware design: schematic capture, PCB layout, design-rule checking and 3D design review.

STRONG CLOSING ANSWER

“I like that this project connects the full engineering process: design the hardware, measure it, troubleshoot differences, analyze the evidence and document what the design can and cannot prove.”

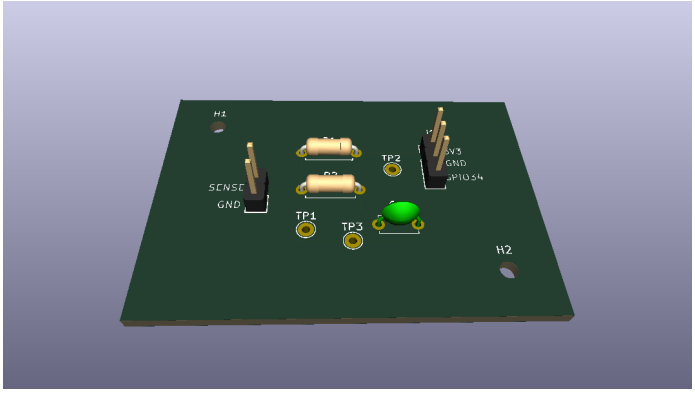
Presentation guide + quick recall

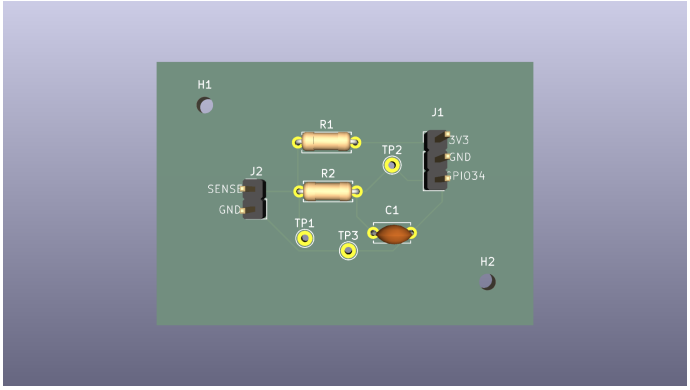
What to show, what to say, and how to close

OPEN THESE SECTIONS IN YOUR FULL ENGINEERING REPORT

Page 4 | Schematic	Explain NTC divider, R1/R2/C1, test points and GPIO34.
Pages 5-6 | PCB / 3D	Explain footprints, routing, 50 × 35 mm layout and KiCad DRC.
Pages 9-10 | Hardware	Explain repeatability and the warming/cooling measurement.
Page 12 | Verification	Explain passed checks, unresolved calibration and PCB-not-built boundary.

YOUR ACTUAL KICAD 3D DESIGN





2- TO 3-MINUTE PRESENTATION FLOW

01. Introduce the goal: a temperature sensor you can measure, log and evaluate.

02. Show the circuit: NTC divider → RC filter → ESP32 GPIO34.

03. Show real evidence: 641 stable samples and a heating/cooling plot.

04. Show the CAD result: schematic, routed PCB, 3D model and DRC checks.

05. Finish with the ADC discrepancy, limitations and next validation step.

FAST MEMORY CUES

DESIGN → BUILD → MEASURE → TROUBLESHOOT → ANALYZE → DOCUMENT → IMPROVE
NTC • GPIO34 • 200 ms • 16-sample average • Python CSV • 0.0412°C SD • 30.829°C peak • 50 × 35 mm PCB • 0 configured DRC violations.